

Task 1 – Containerization: Dockerize a Web Application

Objective

The objective of this task was to containerize a backend web application using Docker and Docker Compose. The application should be able to run inside containers along with its database service while maintaining reproducible and consistent deployments across environments.

Project Overview

For this task, I used my existing Notes API project built in Go.

The Notes API is a RESTful backend application that provides:

- User Registration
- User Authentication using JWT
- Notes CRUD Operations
- PostgreSQL Database Integration
- Database Migrations
- Pagination Support

The application follows a layered architecture consisting of Handlers, Services, Repositories, and PostgreSQL.

Technologies Used

- Go 1.25
 - PostgreSQL 18
 - Docker
 - Docker Compose
 - JWT Authentication
 - Git & GitHub
-

Task Requirements

The following requirements were implemented:

- Dockerfile for application containerization
 - Multi-stage Docker build
 - Docker Compose configuration
 - PostgreSQL database container
 - Health checks
 - Persistent storage using Docker volumes
 - Build and run documentation
-

Dockerfile Configuration

A multi-stage Docker build was implemented.

Build Stage:

- Uses the official Go image
- Downloads project dependencies
- Compiles the application binary

Runtime Stage:

- Uses Alpine Linux
- Copies only the compiled binary
- Produces a lightweight production image

Result:

- Initial image size: ~1.5 GB
- Optimized image size: ~35 MB

Benefits:

- Smaller image size
 - Faster deployments
 - Reduced attack surface
 - Better production readiness
-

Docker Compose Configuration

Docker Compose was used to orchestrate multiple services.

Services Configured:

API Service

Responsibilities:

- Build application image from Dockerfile
- Expose application on port 8080
- Connect to PostgreSQL container
- Load environment variables

PostgreSQL Service

Responsibilities:

- Run PostgreSQL 18 database
 - Expose database port
 - Persist database data using Docker volumes
 - Provide health check validation
-

Health Checks

A PostgreSQL health check was implemented to ensure that the API service starts only after the database is ready to accept connections.

Health Check Command:

`pg_isready -U postgres -d notes`

Benefits:

- Prevents startup race conditions
 - Improves service reliability
 - Ensures successful database connections
-

Docker Volume Configuration

Persistent storage was configured using Docker volumes.

Volume:

postgres_data

Purpose:

- Preserve PostgreSQL data across container restarts
- Maintain database state between deployments

Commands Used

Build Docker Image

`docker build -t notes-api:multistage .`

Build Containers

`docker compose build`

Start Application

`docker compose up`

Run in Detached Mode

`docker compose up -d`

Stop Containers

`docker compose down`

Remove Containers and Volumes

`docker compose down -v`

Verify Running Containers

`docker ps`

Verify API Endpoint

`curl http://localhost:8080/notes`

Container Architecture

Client



Notes API Container



PostgreSQL Container



Docker Volume

Validation

The following validations were performed:

- Docker image builds successfully
 - Multi-stage build completes successfully
 - PostgreSQL container starts successfully
 - Health checks pass successfully
 - API container connects to PostgreSQL
 - Docker networking functions correctly
 - API endpoint is accessible through localhost:8080
 - Persistent volume storage functions correctly
-

Screenshots

Screenshot 1 – Docker Compose Running

docs/screenshots/docker-compose-up.png

Description:

The screenshot demonstrates successful startup of both PostgreSQL and API containers. The database becomes healthy and the API successfully establishes a connection before starting.

Screenshot 2 – API Endpoint Test

docs/screenshots/api-endpoint-test.png

Description:

The screenshot demonstrates successful communication with the API endpoint through the exposed Docker port, confirming that the application is running inside the containerized environment.

Repository Information

GitHub Repository:

<https://github.com/shantam-sharma/Notes-Api>

Release Tag:

v1.1.0

Docker Files:

Dockerfile

docker-compose.yml

.dockerignore

Deployed Endpoint

N/A

The application was containerized and tested locally using Docker Compose. No public deployment was performed as part of this task.

Deliverables

The following deliverables were completed:

- Dockerfile
- Multi-stage Docker build
- Docker Compose configuration
- PostgreSQL container integration
- Health checks
- Project documentation
- Validation screenshots
- Tagged release (v1.1.0)

Conclusion

The Notes API application was successfully containerized using Docker and Docker Compose. The implementation includes a multi-stage Docker build, PostgreSQL containerization, health checks, persistent volumes, and deployment documentation. The final solution satisfies all task requirements and provides a reproducible containerized environment for development and deployment.